

A Project Proposal Report on
Cafe Billing System Using C++

A Project Report Submitted for the Course
Object-Oriented Programming (OOP)
Bachelor of Science in Computer Science

Submitted by:

Bristi Sharma Baral, WU0258526

Date:

08/10/2026



King's College Nepal

Bijulibajar, Kathmandu

ABSTRACT

The Cafe Billing System is a console-based application developed in C++ to help small cafes prepare customer bills quickly and accurately. Many small cafes in Nepal still calculate bills by hand, which takes time and can lead to mistakes in totals, discounts, or daily sales records. This project automates that process: it displays a menu of available items, lets the cashier record the customer's order and quantity, calculates the total automatically, applies a 10% discount when the bill crosses Rs. 1000, prints a receipt, and keeps a running total of the day's sales.

The system is built using Object-Oriented Programming concepts covered in PROG 102, including classes and objects, encapsulation, constructors, and inheritance. Four main classes are used: MenuItem to store item details, Order to handle item selection and bill calculation, Receipt to display the final bill, and Cafe to manage the overall system. A simple Person-to-Customer inheritance relationship is also included to demonstrate how OOP models real-world entities.

The goal of this project is to reduce manual calculation errors, speed up the billing process, and give hands-on practice applying OOP concepts to a real, practical problem. The system can later be extended with features such as file storage for sales records, an admin login, and support for multiple payment methods.

Keywords: *Menu Item, Order, Receipt, Discount, Daily Sales, Object-Oriented Programming*

TABLE OF CONTENTS

Abstract	i
Table of Contents	ii
List of Figures	iii
List of Tables	iv
1 INTRODUCTION	1
1.1 PROBLEM STATEMENT	1
1.2 PROJECT OBJECTIVES	1
1.3 SCOPE AND LIMITATIONS	1
2 LITERATURE REVIEW	3
3 METHODOLOGY	4
3.1 SOFTWARE DEVELOPMENT APPROACH	4
3.2 TOOLS TO BE USED	4
3.3 TECHNOLOGIES TO BE USED	4
4 SYSTEM DESIGN	5
4.1 CLASS OVERVIEW	5
4.2 CLASS DIAGRAM	6
4.3 USE CASE OVERVIEW	7
4.4 SEQUENCE DIAGRAM	8
5 PROPOSED DELIVERABLES	9
6 PROJECT TASK AND TIME SCHEDULE	10
7 FUTURE IMPROVEMENTS	11
Bibliography	12

LIST OF FIGURES

1	UML Class Diagram of the Cafe Billing System	6
2	Use Case Diagram of the Cafe Billing System	7
3	Sequence Diagram of the Billing Process	8

LIST OF TABLES

1	Class Overview	5
2	Project Task and Time Schedule	10

1. INTRODUCTION

This project is a console-based Cafe Billing System built in C++. It is meant to give a small cafe a simple, reliable way to bill customers without doing the math by hand. This section explains the problem the project addresses, its objectives, and its scope.

1.1 PROBLEM STATEMENT

Many small cafes still prepare bills manually. A cashier has to add up item prices, apply a discount if one is available, and write out a receipt by hand. Doing this manually takes time and can easily lead to mistakes, especially during busy hours, and it leaves the cafe owner with no easy way to see how much business was done in a day.

1.2 PROJECT OBJECTIVES

- Automate the billing process so totals and discounts are calculated without manual math.
- Reduce calculation errors compared to a manual, paper-based approach.
- Generate a clear, consistent receipt for every order.
- Keep a running total of daily sales so the cafe owner can track the day's business.
- Apply core Object-Oriented Programming concepts – classes, objects, encapsulation, constructors, and inheritance – in a practical project.

1.3 SCOPE AND LIMITATIONS

The scope of this project is:

- A single cafe with a fixed, predefined menu of items and prices.
- One cashier operates the system per session, through a console (command-line) interface.
- Automatic bill calculation, a flat 10% discount above Rs. 1000, receipt printing, and daily sales tracking.

The limitations of this project are:

- No graphical user interface, the system runs entirely in the console.
- No persistent storage; sales data is kept only for the current run of the program and is not saved to a file or database.
- The menu is fixed in the code and cannot be edited by the user without changing the source.
- Only one payment type (cash) is assumed; no online or card payment handling is included.

2. LITERATURE REVIEW

Manual billing is still common in small cafes and tea shops across Nepal, where the cashier calculates totals by hand or with a basic calculator and writes the receipt on paper or a notebook. This approach is slow, is prone to arithmetic mistakes, and gives the owner no easy record of daily sales without separately tallying receipts at the end of the day.

Commercial Point-of-Sale (POS) systems solve this problem at a larger scale, offering menu management, receipt printing, and sales reporting. However, these systems are usually too complex and too expensive for a very small cafe, and they are not designed as a learning exercise in programming fundamentals.

This project takes the same basic idea behind a POS system – automatic calculation, discounting, and receipt generation – and implements a lightweight, console-based version in C++. Unlike a full commercial POS, it is intentionally simple: no database, no networking, and no graphical interface. This keeps the project appropriately scoped for a first-year OOP course while still solving a real, practical problem for a small cafe.

3. METHODOLOGY

This section describes the approach, tools, and technologies used to build the project.

3.1 SOFTWARE DEVELOPMENT APPROACH

The project will be built incrementally, one feature at a time, so that a working version exists at every stage:

- **Stage 1: Menu display** – define the MenuItem class and print the available items and prices.
- **Stage 2: Order handling** – build the Order class to take item selection and quantity, and compute the subtotal.
- **Stage 3: Discount logic** – apply the 10% discount rule when the subtotal exceeds Rs. 1000.
- **Stage 4: Receipt generation** – build the Receipt class to print a formatted bill.
- **Stage 5: Daily sales tracking** – have the Cafe class accumulate each order's total into a running daily sales figure.

Each stage will be written, compiled, and tested before moving to the next, so errors are caught early.

3.2 TOOLS TO BE USED

- **Code::Blocks / VS Code** – primary editor for writing and running the C++ code.
- **GCC / MinGW compiler** – to compile and run the program.
- **Git and GitHub** – to save progress and keep a version history of the code.

3.3 TECHNOLOGIES TO BE USED

- **C++** – the programming language used to implement the entire system.
- **Standard Template Library (STL)** – for containers such as vectors, used to store menu items and order lines.

4. SYSTEM DESIGN

This section gives an overview of the classes in the system and how a typical billing session flows from menu display to receipt.

4.1 CLASS OVERVIEW

The system is organized around four core classes, plus a small inheritance example:

Table 1: Class Overview

Class	Responsibility	Key Members
MenuItem	Stores details of a single food/drink item	id, name, price
Order	Stores the items and quantities a customer selects; calculates the subtotal	items[], quantities[], calculateTotal()
Receipt	Formats and prints the final bill	printReceipt(order, discount)
Cafe	Runs the overall system: shows the menu, takes orders, tracks daily sales	menu[], dailySales, displayMenu(), addSale()
Person → Customer	Simple inheritance example: Customer inherits basic identity fields from Person	name, contact (inherited); orderHistory (Customer)

MenuItem, Order, Receipt, and Cafe use encapsulation (private data with public functions) and constructors to initialize objects, which are the main OOP concepts this project is meant to demonstrate.

4.2 CLASS DIAGRAM

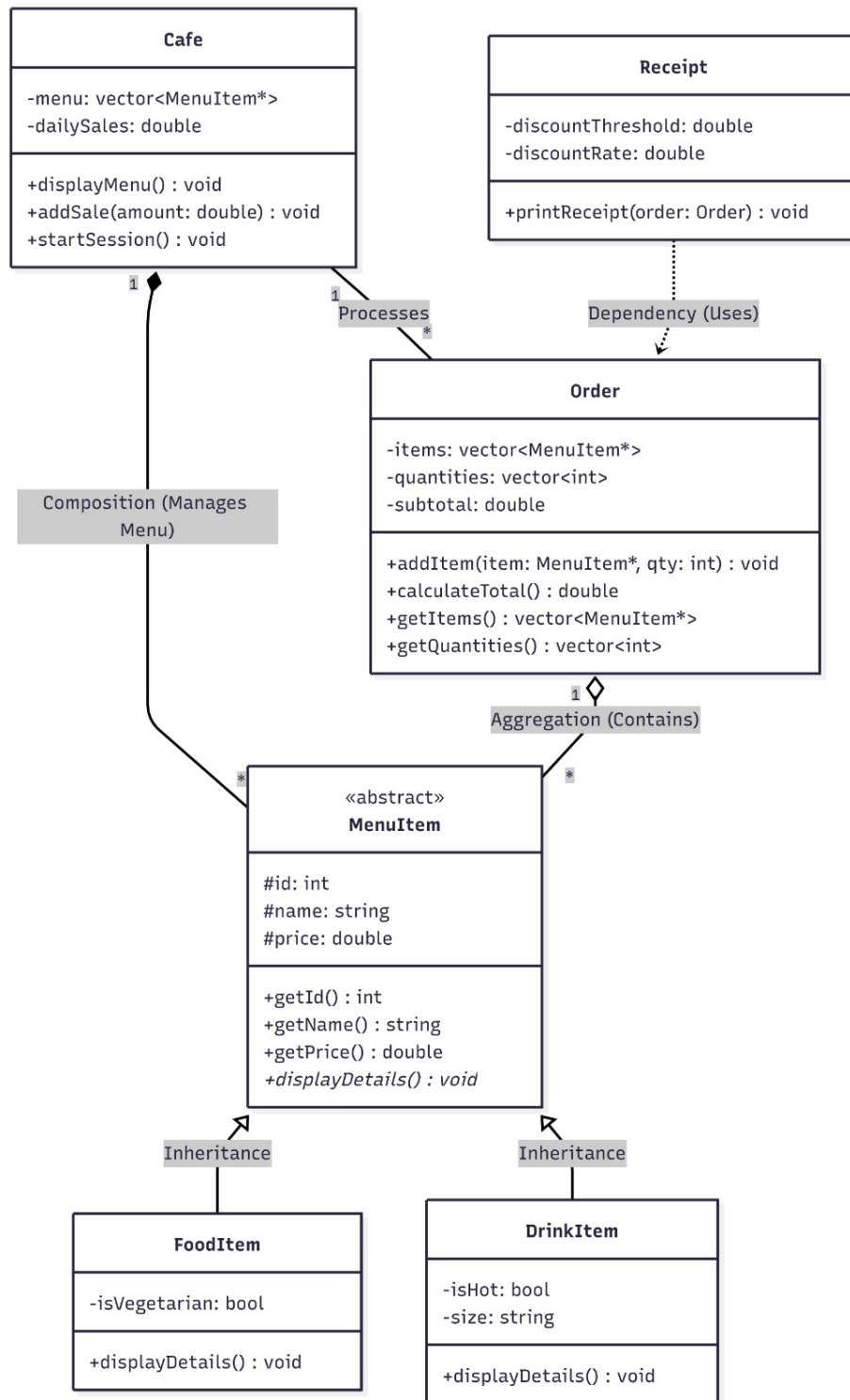


Figure 1: UML Class Diagram of the Cafe Billing System

4.3 USE CASE OVERVIEW

The system has a single actor, the Cashier, who interacts with the following use cases:

- **View Menu** – the cashier looks up available items and prices.
- **Place Order** – the cashier selects items and enters quantities for a customer.
- **Calculate Bill (includes Apply Discount)** – the system computes the subtotal and applies the discount if it qualifies.
- **Print Receipt** – the system displays the finalized bill for the customer.
- **Update Daily Sales** – the system adds the completed order's total to the day's running sales figure.

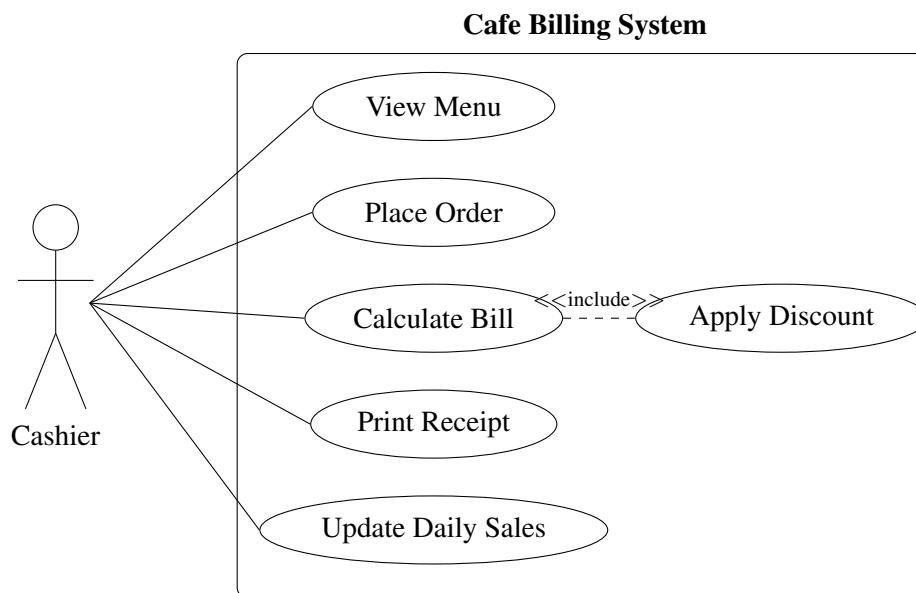


Figure 2: Use Case Diagram of the Cafe Billing System

4.4 SEQUENCE DIAGRAM

The sequence diagram below illustrates the flow of a typical billing transaction, from the cashier viewing the menu through to the receipt being printed and the daily sales figure being updated.

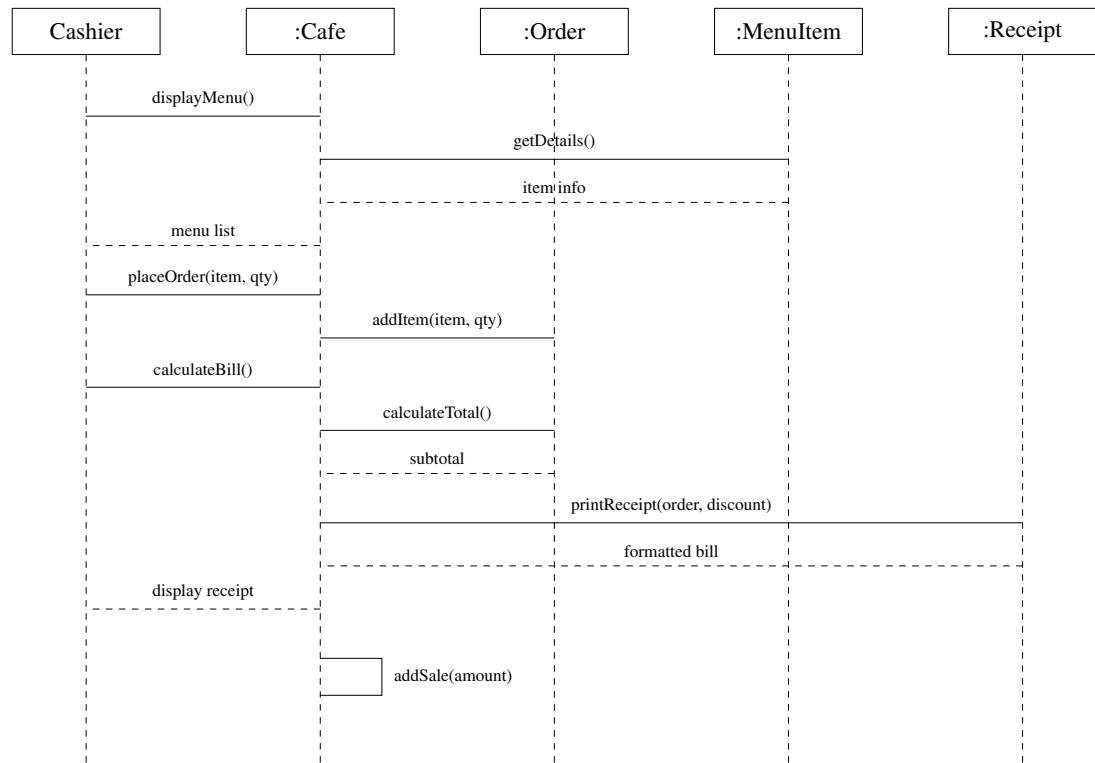


Figure 3: Sequence Diagram of the Billing Process

5. PROPOSED DELIVERABLES

This project will deliver a single-user, console-based C++ application with the following functionality:

- A menu display showing all available items and their prices.
- An order-entry flow where the cashier selects items and quantities.
- Automatic bill calculation, including the 10% discount rule for bills above Rs. 1000.
- A printed receipt showing items, quantities, subtotal, discount, and final payable amount.
- A running total of daily sales, viewable at the end of a session.
- Source code organized into separate class files (MenuItem, Order, Receipt, Cafe, Person/Customer), along with this proposal and a short project report.

6. PROJECT TASK AND TIME SCHEDULE

The project is scoped to be completed over roughly two weeks, as broken down below.

Table 2: Project Task and Time Schedule

TASK	APPROX. DURATION (DAYS)
Requirement analysis and class planning	1
MenuItem and menu display	1
Order class and bill calculation	2
Discount logic	1
Receipt class and formatting	1
Daily sales tracking (Cafe class)	1
Person/Customer inheritance example	1
Testing and debugging	2
Documentation and proposal/report writeup	2

7. FUTURE IMPROVEMENTS

If extended beyond the course requirements, the system could add:

- Saving daily sales and receipts to a file so records persist between runs.
- An admin login to edit menu items and prices.
- A search function to look up menu items quickly.
- Support for multiple payment methods (cash, eSewa, Khalti).
- A simple customer database for repeat customers or loyalty discounts.

BIBLIOGRAPHY

- Asrani, H., Vishwakarma, S., Asrani, D., & Asrani, K. (2024). Point of sale systems: A review. *International Journal of Innovative Research in Computer Science & Technology*, 12(Special Issue-1). <https://ijircst.org/DOC/63-point-of-sale-systems.pdf>
- Sison, P., Oreiro, K., Camalit, J., & Ng, G. (2019). A problem oriented approach to implementing an inventory and point-of-sale system for company KCP. *Innovatus*, 2(1), 95–100. https://innovatus-pub.github.io/abstractpublications_archive/2019
- Zade, A., & Adkar, P. (2026). Smart POS system for efficient sales and inventory management in small businesses. *IRE Journals*, 9(11), 2855–2860. <https://doi.org/10.64388/IREV9I11-1717941>
- cppreference.com. (n.d.). *C++ reference*. Retrieved August 9, 2026, from <https://en.cppreference.com/>